

Piotr's TechBlog

Java, Spring, Kotlin, microservices, Kubernetes, containers

Using RAG and Vector Store with Spring AI

AI, Spring Boot

Using RAG and Vector Store with Spring AI

By [piotr.minkowski](#) • February 24, 2025 •  5

Search

Follow Blog via Email

Enter your email address to follow this blog and receive notifications of new



This article will teach you how to create a Spring Boot application that uses RAG (Retrieval Augmented Generation) and vector store with Spring AI. We will continue experiments with stock data, which were initiated in my previous article about Spring AI. This is the third part of my series of articles about Spring Boot and AI. It is worth reading the following posts before proceeding with the current one:

1. <https://piotrminkowski.com/2025/01/28/getting-started-with-spring-ai-and-chat-model>: The first tutorial introduces the Spring AI project and its support for building applications based on chat models like OpenAI or Mistral AI.
2. <https://piotrminkowski.com/2025/01/30/getting-started-with-spring-ai-function-calling>: The second tutorial shows Spring AI support for Java function calling with the OpenAI chat model.

This article will show how to include one of the vector stores supported by Spring AI and advisors dedicated to RAG support in our sample application codebase used by two previous articles. It will connect to Open AI API, but you can easily switch to other models using Mistral AI or Ollama support in Spring AI. For more details, please refer to my first article.

Source Code

If you would like to try it by yourself, you may always take a look at my source code. To do that, you must clone my sample GitHub [repository](#). Then you should only follow my instructions.

Motivation for RAG with Spring AI

posts by email.

Enter your email address

Follow

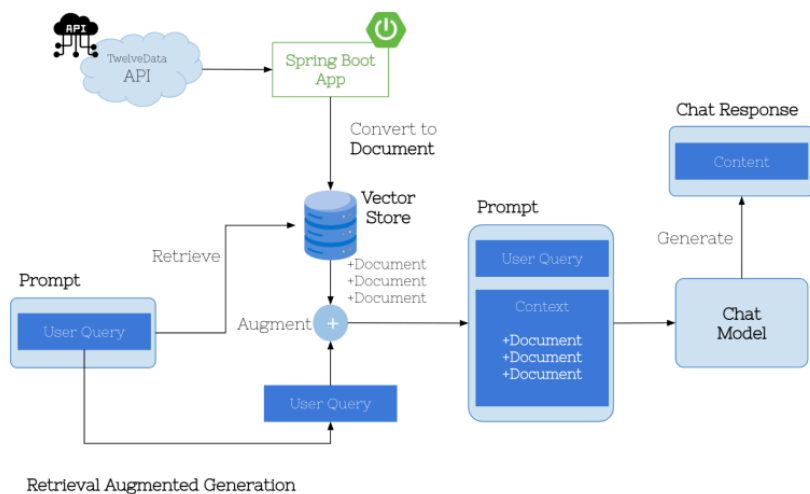
Categories

- AI (18)
- Cloud (42)
- Containers (67)
- Continuous Integration (37)
- Data Grids (7)
- Kotlin (9)
- Kubernetes (115)
- Message Brokers (27)
- Micronaut (12)
- Microservices (64)
- Other (28)
- Performance (28)
- Quarkus (18)
- Security (26)
- Spring Boot (113)

The problem to solve is similar to the one described in my previous article about the Spring AI function calling feature. Since the OpenAI model is trained on a static dataset it does not have direct access to the online services or APIs. We want it to analyze stock growth trends for the biggest companies in the US stock market. Therefore, we must obtain share prices from a public API that returns live stock market data. Then, we can store this data in our local database and integrate it with the sample Spring Boot AI application. Instead of a typical relational database, we will use a vector store. In vector databases, queries work differently from those in traditional relational databases. Instead of looking for exact matches, they conduct similarity searches. It retrieves the most similar vectors to a given input vector.

After loading all required data into a vector database, we must integrate it with the AI model. Spring AI provides a comfortable mechanism for that based on the **Advisors** API. We have already used some built-in advisors in the previous examples, e.g. to print detailed AI communication logs or enable chat memory. This time they will allow us to implement a Retrieval Augmented Generation (RAG) technique for our app. Thanks to that, the Spring Boot app will retrieve similar documents that best match a user query before sending a request to the AI model. These documents provide context for the query and are sent to the AI model alongside the user's question.

Here's a simplified visualization of our process.



Vector Store with Spring AI

○ Spring Cloud (51)

○ Uncategorized (1)

Tags

Apache Kafka

argocd

Consul

Continuous Delivery

Continuous Integration

Docker

Eureka

gitops

Helm

istio

Java

Jenkins

Jib

JPA

kafka

kind

Knative

Kotlin

Kubernetes

Maven

Micronaut

microservices

Minikube

OAuth2

open-ai

openshift

Performance

postgresql

Prometheus

quarkus

Reactor

REST

security

Skaffold

Spring

spring-ai

Spring Boot

Spring Boot Actuator

Spring Cloud

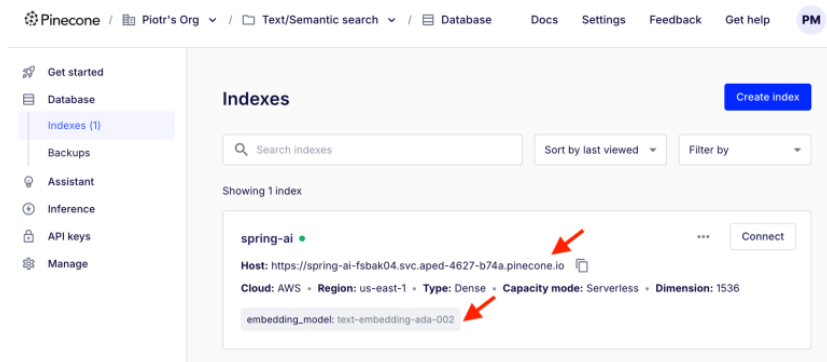
Spring Cloud Gateway

Spring Cloud Kubernetes

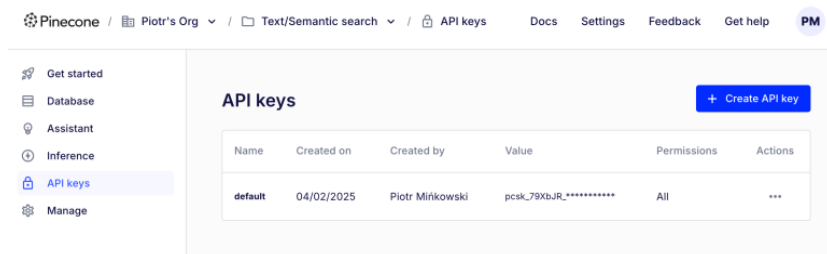
Set up Pinecone Database

In this section, we will prepare a vector store, integrate it with our Spring Boot application, and load some data there. Spring AI supports various vector databases. It provides the **VectorStore** interface to directly interact with a vector store from our Spring Boot app. The full list of supported databases can be found in the Spring AI docs [here](#).

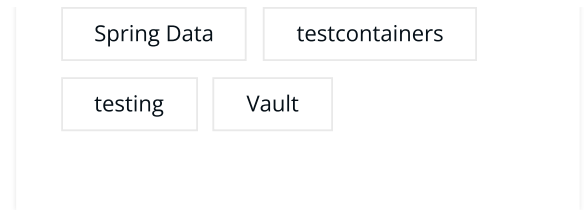
We will proceed with the Pinecone database. It is a popular cloud-based vector database, that allows us to store and search vectors efficiently. Instead of a cloud-based database, we can set up a local instance of another popular vector store – ChromaDB. In that case, you can use the `docker-compose.yml` file in the repository root directory, to run that database with the `docker compose up` command. With Pinecone we need to sign up to create an account on their portal. Then we should create an index. There are several customizations available, but the most important thing is to choose the right embedding model. Since `text-embedding-ada-002` is a default embedding model for OpenAI we should choose that option. The name of our index is `spring-ai`. We can read an environment and project name from the generated host URL.



After creating an index, we should generate an API key.



Then, we will copy the generated key and export it as the `PINECONE_TOKEN` environment variable.



```
export PINECONE_TOKEN=<YOUR_PINECONE_APIKEY>
```

Integrate Spring Boot app with Pinecode using Spring AI

Our Spring Boot application must include the `spring-ai-pinecone-store-spring-boot-starter` dependency to smoothly integrate with the Pinecone vector store.

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-pinecone-store-spring-boot-st
</dependency>
```

Then, we must provide connection settings and credentials to the Pinecone database in the Spring Boot `application.properties` file. It must be at least the Pinecode API key, environment name, project name, and index name.

```
spring.ai.vectorstore.pinecone.apiKey = ${PINECONE_TO
spring.ai.vectorstore.pinecone.environment = aped-462
spring.ai.vectorstore.pinecone.projectId = fsbak04
spring.ai.vectorstore.pinecone.index-name = spring-ai
```

After providing all the required configuration settings we can inject and use the `VectorStore` bean e.g. in our application REST controller. In the following code fragment, we load input data into a vector store and perform a simple similarity search to find the most growth stock trend. We individually query the Twelvedata API for each company from a list and deserialize the response to the `StockData` object. Then we create a Spring AI `Document` object, which contains the name of a company and share close prices for the last 10 days. Data is written in the JSON format.

```
@RestController
@RequestMapping("/stocks")
public class StockController {

    private final ObjectMapper mapper = new ObjectMap
    private final static Logger LOG = LoggerFactory.g
    private final RestTemplate restTemplate;
```

```

private final VectorStore store;

@Value("${STOCK_API_KEY}")
private String apiKey;

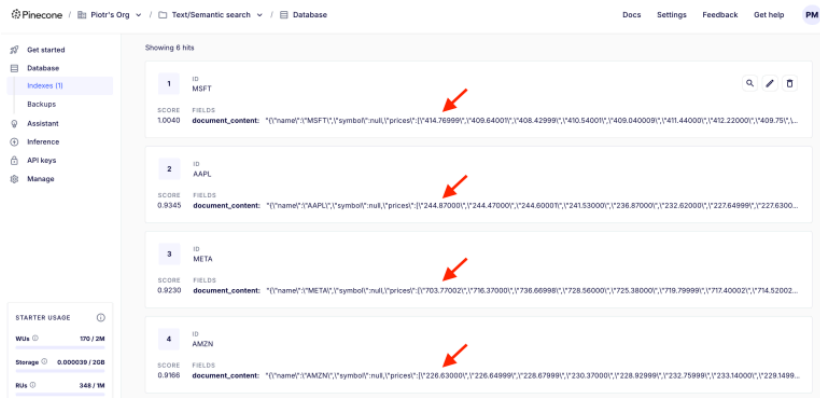
public StockController(VectorStore store,
                       RestTemplate restTemplate) {
    this.store = store;
    this.restTemplate = restTemplate;
}

@PostMapping("/load-data")
void load() throws JsonProcessingException {
    final List<String> companies = List.of("AAPL"
    for (String company : companies) {
        StockData data = restTemplate.getForObject(
            StockData.class,
            company,
            apiKey);
        if (data != null && data.getValues() != null) {
            var list = data.getValues().stream().
            var doc = Document.builder()
                .id(company)
                .text(mapper.writeValueAsString(list))
                .build();
            store.add(List.of(doc));
            LOG.info("Document added: {}", company);
        }
    }
}

@GetMapping("/docs")
List<Document> query() {
    SearchRequest searchRequest = SearchRequest.builder()
        .query("Find the most growth trends")
        .topK(2)
        .build();
    List<Document> docs = store.similaritySearch(searchRequest);
    return docs;
}
}

```

Once we start our application and call the **POST /stocks/load-data** endpoint, we should see 6 records loaded into the target store. You can verify the content of the database in the Pinocone index browser.



Then we can interact directly with a vector store by calling the **GET** `/stocks/docs` endpoint.

```
curl http://localhost:8080/stocks/docs
```

Implement RAG with Spring AI

Use QuestionAnswerAdvisor

Previously we loaded data into a target vector store and performed a simple search to find the most growth trend. Our main goal in this section is to incorporate relevant data into an AI model prompt. We can implement RAG with Spring AI in two ways with different advisors. Let's begin with **QuestionAnswerAdvisor**. To perform RAG we must provide an instance of **QuestionAnswerAdvisor** to the **ChatClient** bean. The **QuestionAnswerAdvisor** constructor takes the **VectorStore** instance as an input argument.

```
@RequestMapping("/v1/most-growth-trend")
String getBestTrend() {
    PromptTemplate pt = new PromptTemplate("""
        {query}.
        Which {target} is the most % growth?
        The 0 element in the prices table is the
        """);

    Prompt p = pt.create(
        Map.of("query", "Find the most growth tre
              "target", "share")
    );

    return this.chatClient.prompt(p)
        .advisors(new QuestionAnswerAdvisor(store
```

```
.call()
.content();
```

```
}
```

Then, we can call the endpoint `GET /stocks/v1/most-growth-trend` to see the AI model response. By the way, the result is not accurate.

```
curl http://localhost:8080/stocks/v1/most-growth-trend
To find the most growth trends, we can calculate the percentage growth for each stock
based on the latest price (0 element) and the oldest price (last element).

1. **AAPL**
  - Latest price: 244.87000
  - Oldest price: 232.62000
  - Percentage growth: ((244.87000 - 232.62000) / 232.62000) * 100 = 5.27%

2. **MSFT**
  - Latest price: 414.76999
  - Oldest price: 413.29001
  - Percentage growth: ((414.76999 - 413.29001) / 413.29001) * 100 = 0.36%

3. **AMZN**
  - Latest price: 226.63000
  - Oldest price: 236.17000
  - Percentage growth: ((226.63000 - 236.17000) / 236.17000) * 100 = -4.05%

4. **META**
  - Latest price: 703.77002
  - Oldest price: 704.87000
  - Percentage growth: ((703.77002 - 704.87000) / 704.87000) * 100 = -0.15%

Based on these calculations, **AAPL** has the most percentage growth at 5.27%.
```

Let's work a little bit on our previous code. We will publish a new version of the AI prompt under the `GET /stocks/v1-1/most-growth-trend` endpoint. The changed lines have been highlighted. We build the `SearchRequest` objects that return the top 3 records with the `0.7` similarity threshold. The newly created `SearchRequest` object must be passed as an argument in the `QuestionAnswerAdvisor` constructor.

```
@RequestMapping("/v1-1/most-growth-trend")
String getBestTrendV11() {
    PromptTemplate pt = new PromptTemplate("""
        Which share is the most % growth?
        The 0 element in the prices table is the
        Return a full name of company instead of
        """);

    SearchRequest searchRequest = SearchRequest.builder()
        .query("""
            Find the most growth trends.
            The 0 element in the prices table is the
            """)
        .topK(3)
        .similarityThreshold(0.7)
        .build();

    return this.chatClient.prompt(pt.create())
        .advisors(new QuestionAnswerAdvisor(store
            .call()
```



```
.content();
```

```
}
```

Now, the results are more accurate. The model also returns the full name of companies instead of a market shortcut.

```
curl http://localhost:8080/stocks/v1-1/most-growth-trend
To determine the share with the most percentage growth, we need to calculate the percentage change for each company based on their prices.

1. **Microsoft Corporation (MSFT)**:
  - Latest Price: $414.76999
  - Oldest Price: $409.64001
  - Percentage Growth = ((414.76999 - 409.64001) / 409.64001) * 100 ≈ 1.03%

2. **Apple Inc. (AAPL)**:
  - Latest Price: $244.87000
  - Oldest Price: $232.47000
  - Percentage Growth = ((244.87000 - 232.47000) / 232.47000) * 100 ≈ 5.34%

3. **NVIDIA Corporation (NVDA)**:
  - Latest Price: $139.23000
  - Oldest Price: $124.83000
  - Percentage Growth = ((139.23000 - 124.83000) / 124.83000) * 100 ≈ 11.53%

Based on these calculations, **NVIDIA Corporation** has the highest percentage growth at approximately **11.53%**.
```

Use RetrievalAugmentationAdvisor

Instead of the `QuestionAnswerAdvisor` class, we can also use the experimental `RetrievalAugmentationAdvisor`. It provides an out-of-the-box implementation for the most common RAG flows, based on a modular architecture. There are several built-in modules we can use with `RetrievalAugmentationAdvisor`. We will include the `RewriteQueryTransformer` module that uses LLM to rewrite a user query to provide better results when querying a target vector database. It requires the `query` and `target` placeholders to be present in the prompt template. Thanks to that transformer we can retrieve the optimal set of records for a percentage growth calculation.

```
@RestController
@RequestMapping("/stocks")
public class StockController {

    private final ObjectMapper mapper = new ObjectMapper()
    private final static Logger LOG = LoggerFactory.getLogger(StockController.class)
    private final ChatClient chatClient;
    private final RewriteQueryTransformer.Builder rqt;
    private final RestTemplate restTemplate;
    private final VectorStore store;

    @Value("${STOCK_API_KEY}")
    private String apiKey;

    public StockController(ChatClient.Builder chatClientBuilder, VectorStore store, RewriteQueryTransformer.Builder rqt, RestTemplate restTemplate, ObjectMapper mapper) {
        this.chatClient = chatClientBuilder.build();
        this.store = store;
        this.rqt = rqt;
        this.restTemplate = restTemplate;
        this.mapper = mapper;
    }

    @GetMapping
    public List<Company> getMostGrowth() {
        String query = "Which company has the highest percentage growth?";
        String target = "company";
        List<Record> records = store.search(query, target);
        List<Company> companies = new ArrayList<>();
        for (Record record : records) {
            Company company = mapper.convertValue(record.getContent(), Company.class);
            companies.add(company);
        }
        return companies;
    }
}
```

```

        VectorStore store,
        RestTemplate restTemplate)
this.chatClient = chatClientBuilder
    .defaultAdvisors(new SimpleLoggerAdvi
    .build());
this.rqtBuilder = RewriteQueryTransformer.bui
    .chatClientBuilder(chatClientBuilder)
this.store = store;
this.restTemplate = restTemplate;
}

```

// other methods ...

```

@RequestMapping("/v2/most-growth-trend")
String getBestTrendV2() {
    PromptTemplate pt = new PromptTemplate("""
        {query}.
        Which {target} is the most % growth?
        The 0 element in the prices table is
        """);

    Prompt p = pt.create(Map.of("query", "Find th

    Advisor retrievalAugmentationAdvisor = Retrie
        .documentRetriever(VectorStoreDocumen
        .similarityThreshold(0.7)
        .topK(3)
        .vectorStore(store)
        .build())
        .queryTransformers(rqtBuilder.promptT
        .build());

    return this.chatClient.prompt(p)
        .advisors(retrievalAugmentationAdviso
        .call()
        .content();
}

```

```

}

```

Once again, we can verify the AI model response by calling the **GET** **/stocks/v2/most-growth-trend** endpoint. The response is similar to those generated by the **GET /stocks/v1-1/most-growth-trend** endpoint.

```

curl http://localhost:8080/stocks/v2/most-growth-trend
st Price}} \right) \times 100
\}

1. **MSFT**:
- Latest Price: 414.76999
- Oldest Price: 413.29001
- Growth:
\left( \frac{414.76999 - 409.64001}{409.64001} \right) \times 100 \approx 1.00\%
\}

2. **NVDA**:
- Latest Price: 139.23000
- Oldest Price: 124.83000
- Growth:
\left( \frac{139.23000 - 124.83000}{124.83000} \right) \times 100 \approx 11.53\%
\}

3. **AAPL**:
- Latest Price: 244.87000
- Oldest Price: 232.47000
- Growth:
\left( \frac{244.87000 - 227.63000}{227.63000} \right) \times 100 \approx 7.60\%
\}

The stock with the most percentage growth is **NVDA**, with approximately **11.53%** growth.

```

Run the Application

Only to remind you. Before running the application, we should provide OpenAI and TwelveData API tokens.

```

$ export OPEN_AI_TOKEN=<YOUR_OPEN_AI_TOKEN>
$ export STOCK_API_KEY=<YOUR_STOCK_API_KEY>
$ mvn spring-boot:run

```

Final Thoughts

In this article you learned how to use an important AI technique called Retrieval Augmented Generation (RAG) with Spring AI. Spring AI simplifies RAG by providing built-in support for vector stores and easy data incorporation into the chat model through the **Advisor** API. However, since **RetrievalAugmentationAdvisor** is an experimental feature we cannot rule out some changes in future releases.

[LLM](#)
[open-ai](#)
[pinecone](#)
[RAG](#)
[Spring Boot](#)
[spring-ai](#)
[vector store](#)



Written by:

piotr.minkowski

[View All Posts →](#)

5 COMMENTS



zcz February 27, 2025 6:33 am

Hi, Piotr. I'm also exploring similar frameworks. I've developed a lightweight framework called JLangChain, <https://github.com/flower-trees/j-langchain>. I'd be delighted to have you use it, or provide valuable suggestions and engage in discussions. Looking forward to your reply.

Loading...

[REPLY](#)



piotr.minkowski March 3, 2025 12:48 pm

Hi. It looks interesting. I'll definitely take a look on.

Loading...

[REPLY](#)



Alassane March 1, 2025 12:43 pm

Very nice article.

I have a question regarding RAG, how the chat model "understands" the data coming from the vector database ? Are thoses data specific for each

vector engine or is this standard ?

Best regards

Loading...

[REPLY](#)



piotr.minkowski March 3, 2025 12:46 pm

Vector database is integrated with the model through SpringAI. If you insert data into a vector store it generate embeddings using LLM and put the result into a vectore store.

Loading...

[REPLY](#)



Alassane March 3, 2025 12:59 pm

Great, so under the hood, the vector database is connected to the configured LLM ?

Loading...

[REPLY](#)

Leave a Reply

Contact info

If you would like to contact me in order you have any questions, thoughts or ideas (e.g. suggestions for future articles) contact me via email.

 Warsaw, Poland
piotr.minkowski@gmail.com

Top Posts & Pages

-  [Claude Code Template for Spring Boot](#)
-  [Startup CPU Boost in Kubernetes with In-Place Pod Resize](#)
-  [Part 1: Creating microservice using Spring Cloud, Eureka and Zuul](#)
-  [Create Apps with Claude Code on Ollama](#)

Social Media

○ Deep Dive into Kafka Offset
Commit with Spring Boot